

非结构化数据工厂

L01 · Parse

IDP 替代抽文本
LlamaParse + Marker

L02 · Chunk

结构感知 + 代码 AST
Late + Contextual

L03 · Evidence

4 levels 证据链
Citations API GA

L04 · Quality

质检 + Drift
CI 门禁 + 增量更新

L05 · Multimodal

Vision LLM + CLIP
Whisper + Gemini 视频



LESSON 01 · 智能文档解析

L01 从「抽正文」走到「保结构」

这是 RAG 翻车的第一大根源 · 做过 RAG 的都熟悉

真实场景 · 财报 PDF 给 LLM,问数字答得乱七八糟

前几年我做客户的金融 RAG · 团队用 PyPDF 抽文本 · Demo 跑得溜 · 上线第一周客户问「Q3 净利润多少」,LLM 答了一个 Q2 的数字。排查 6 小时 · 根因是 PyPDF 把 2 栏排版的财报抽成单栏 · Q3 表格的数字串到 Q2 表格旁边了 · LLM 看不懂上下文,瞎填。那不是模型笨,是「文档不是字符串」 · 你不保结构,LLM 永远不可能答对。

咱们这节聊 5 件事：

- 为什么「文档 ≠ 字符串」—— IDP 才是 2026 的事实标准
- 5 阶段 pipeline · Load → Layout → Extract → Normalize → Persist · 缺一阶段就盲
- PDF 5 大难点 + 类型识别 · Marker / LlamaParse / Mistral OCR 2026 工具栈
- 表格处理 3 大难点 + 3 种存储策略 · 比纯文本难 10 倍
- Docling 实战代码 + 工具横评

DOCUMENT ≠ STRING

文档不是字符串 · 这是 RAG 工程的第一性问题

同样一份 PDF · 有结构 vs 没结构 · 下游 RAG 效果差 3-5 倍

维度	把 PDF 当字符串	把 PDF 当结构化资产	差异
多栏 layout	抽成单栏 / 文字串行	识别 column · 各栏分别处理	财报必坏 → 必对
表格	抽成几列空格分隔的乱码	markdown table / cell 边界保留	LLM 看得懂 → 看不懂
标题层级	全是 plain text	H1 / H2 / H3 + chunk metadata	检索精度 +30%
图表 / 公式	直接丢	caption + LaTeX + 引用 ref	专业 RAG 必须
页眉页脚	混入正文搞乱 chunk	识别 + 剥离 + 入 metadata	幻觉率 -40%
坐标信息	没有	每个 chunk 带 page + bbox	可点击高亮原文

老司机说 老司机口诀 · PDF 处理工程化 = 拒绝「文字提取」 · 必须是「结构提取」 · 否则 RAG 答案永远飘



OLD VS NEW

纯文本抽取 vs IDP · 本质差别

维度	旧:PyPDF / pdftotext	新:IDP (LlamaParse / Marker / Docling)	2026
多栏识别	✗ 串行	✓ Layout-aware	必备
表格 → markdown	✗ 乱码	✓ markdown table 保 cell	必备
标题层级 H1/H2/H3	✗ 没有	✓ 识别 + metadata	必备
公式 → LaTeX	✗ 抽成方框	✓ MathML / LaTeX	专业 RAG
图表 → 描述	✗ 直接丢	✓ VLM caption	主流
坐标 bbox + page	✗ 没有	✓ Citations / 高亮支撑	To B 必备
扫描版 PDF	✗ 不支持	✓ OCR 自动 fallback	主流
速度	几页/秒	0.5-2 页/秒(SaaS)	足够
价格	\$0	\$0.003-0.01 / 页	可控

5-STAGE PIPELINE

文档处理 5 阶段流水线 · 每阶段独立工程对象

Load → Layout → Extract → Normalize → Persist · 缺一阶段就有盲区

Load	Layout	Extract	Normalize	Persist
加载 + 类型识别 mimetype / 编码 / 扫描 vs 文本型 / 多页文档拆分	布局检测 多栏 / 表格 / 图 / 页眉页脚 / 标题层级 H1-H6	内容抽取 文字 / 表格 → markdown / 公式 → LaTeX / 图 → caption	清洗 + 结构化 去噪 / 合并断行 / chunk 划分 / metadata 标注	入库 + 版本 原文档 + chunks + bbox + version_id 一起持久化

老司机说

5 阶段是「最小不可分」· 少一个就有盲区。我做过的项目都按这个拆· 每个阶段 1 个工程对象 · 独立可测 / 可换工具 / 可监控。
Load 错了· 后面全错 · 所以这阶段优先级最高 · PDF 类型识别 = 整个 pipeline 的入口

LAYOUT MATTERS

布局信息为什么是 RAG 的关键 · 不是好看,是语义

这 3 个东西丢了 · 下游 RAG 必坏

多栏 column

- 财报 / 论文 / 合同
- 抽成单栏 → 跨栏串扰
- 净利率串到营收旁边
- LLM 看到一堆乱数字
- 识别 · PyMuPDF blocks_xy 或 IDP

标题层级 H1-H6

- "3.2.1 节 / Article 5" 这种
- 丢掉 → chunk 失去上下文
- 保留 → metadata 入检索
- "找 5 章利润分析" 直接命中
- 识别 · 字号 + 缩进 + 编号 pattern

表格 cell 边界

- 股价表 / 财报表 / 价格表
- 抽成空格分隔 → LLM 不识
- 保 markdown 或 cell 坐标
- 表格描述 + markdown 双轨
- 识别 · LlamaParse / Camelot

关键判断

判断 · Layout 不是 UI 问题 · 是「让 LLM 看得懂」的语义问题

PDF 5 难点

PDF 处理的 5 大难点 · + 类型识别 · 入口对了一半工作就稳了

60-80% 企业知识在 PDF · 而 PDF 是企业最大的工程黑洞

5 大难点

- Layout 复杂(多栏 / 表 / 图)
- 扫描版没文字层 → 必须 OCR
- 表格 / 公式难保留语义
- 页眉页脚 / 水印混入
- 字体编码乱(老 PDF)

类型识别 · 入口

- text-based · coverage > 0.8
- scanned · coverage < 0.2
- hybrid · 介于中间
- PyMuPDF 跑一遍判即可
- 走对工具能省 70% 成本

Layout 还原要素

- Title hierarchy (H1/H2/H3)
- 段落分割(不是按行)
- 表格 → markdown table
- 图 → 描述 + 引用
- 公式 → LaTeX

```
import fitz # PyMuPDF · PDF 类型识别 30 行代码
def detect_pdf_type(path):
    doc = fitz.open(path); has_text = sum(1 for p in doc if p.get_text().strip()) / len(doc)
    return "text-based" if has_text > 0.8 else "scanned" if has_text < 0.2 else "hybrid"
# text-based → LlamaParse / Marker · scanned → Mistral OCR / Azure DI · hybrid → 双管 fallback
```

PDF TOOLKIT 2026

PDF 处理工具栈 2026 · 5 家主流横评

开源 Marker 占 50% · 企业 LlamaParse 占 30% · Mistral OCR 占 20%

工具	类型	关键特性 2026	价格	适用
Marker	开源 (MIT)	Layout-aware + 表格 + 公式 → markdown · MIT 协议	\$0	开源首选 ★
LlamaParse	商业 SaaS	复杂 layout / 合同 / 多语言 · LangChain 集成	\$0.003/页	企业首选 ★
Mistral OCR	商业 SaaS · 2026 新	精度 > AWS Textract 5-10% · 便宜 30%	\$0.002/页	便宜大量 ★
Docling	开源 (IBM)	Granite-Docling 2025.9 · 表格强 · 公式 LaTeX	\$0	中文友好
Azure Doc Intel	商业 SaaS	老牌企业级 · 合规 / 表单识别	\$0.01/页	Azure 用户
AWS Textract	商业 SaaS	美企老牌 · 表格 + Form 强	\$0.015/页	AWS 重度
Unstructured	开源 + SaaS	Hi-Res 模式接 layout-parser	\$0 / 商业版	通用 fallback

REAL ARTIFACT · MARKER

Marker 实战 · 开源最强 PDF → markdown 库

MIT 协议 · GitHub 19k+ stars · 2024-2026 开源 PDF 处理几乎默认

```
# pip install marker-pdf
from marker.converters.pdf import PdfConverter
from marker.models import create_model_dict

converter = PdfConverter(
    artifact_dict=create_model_dict(),
    config={
        "extract_images": True,      # 图片单独抽
        "output_format": "markdown",
        "use_llm": True,             # LLM 兜底疑难
        "llm_service": "anthropic",  # Claude Sonnet 4.5
        "force_ocr": False,          # text-based 不必 OCR
        "paginate_output": True,     # 保留 page break
    },
)
rendered = converter("./financial_report_2025_Q3.pdf")

markdown_text = rendered.markdown
images = rendered.images      # {page_n: PIL.Image}
metadata = rendered.metadata  # title / authors / page_count

# 拿到的 markdown 已经处理好:
# · 多栏 → 自动分栏识别后串接
# · 表格 → | header | header | 标准 markdown
# · 公式 → $$LaTeX$$
# · 标题 → # / ## / ###
# · 图 → ![desc](images/img_001.png)

with open("output.md", "w") as f:
    f.write(markdown_text)
```

老司机说

Marker 工程要点 5 条

- LLM 兜底 · use_llm=True
- 复杂 layout 走 Claude 重排
- extract_images=True
- 图独立存,后面接 VLM caption
- paginate_output=True
- 保 page break · evidence 用
- force_ocr=False
- 先看 PDF 类型再决定 OCR
- 输出标准 markdown
- 下游 chunker 直接吃
- 比自研抽取器节省 70% 代码量

TOOL LANDSCAPE 2025

IDP 工具横评 · 按规模 + 精度 4 个梯队

开源做主干 + VLM 兜底疑难 + 商业 API 只在合规场景

梯队	工具	强项	弱项	适用
T1 开源主流	Marker / Docling	Layout + 表格 + 公式 · MIT	复杂合同需 LLM 兜底	主干 ★
T1 商业 SaaS	LlamaParse	复杂 layout / multi-lang / LangChain	API 费用	To B 企业 ★
T1 新势力	Mistral OCR 2026	便宜 + 精度高 · 多语言	新工具生态待补	量大场景
T2 VLM 兜底	Claude Sonnet 4.5 + 截图	解决 layout 极端 case	贵 · \$0.05/页	5% 疑难
T2 老牌商业	Azure DI / Texttract	企业 SLA + 合规	价高 + 锁定 cloud	合规场景
T3 学术	GROBID / Nougat	学术论文 / 引用解析	通用差	论文 RAG

REAL ARTIFACT · DOCLING

Docling 实战 · IBM 出品 · Granite-Docling 2025.9 强化版

中文友好 · 公式强 · 完全开源 · 跟 Marker 互补的选择

```
# pip install docling
from docling.document_converter import DocumentConverter
from docling.datamodel.base_models import InputFormat
from docling.datamodel.pipeline_options import PdfPipelineOptions

opts = PdfPipelineOptions(
    do_ocr=False,          # 先看类型
    do_table_structure=True, # 表格 cell 还原
    table_structure_options={
        "do_cell_matching": True, # cell 边界识别
        "mode": "accurate",      # accurate / fast
    },
    images_scale=2.0,      # 图像 2x · 后续 VLM 看更清
    generate_picture_images=True,
)

converter = DocumentConverter(
    format_options={InputFormat.PDF: opts}
)
result = converter.convert("./contract_2025.pdf")
doc = result.document

# 5 种导出格式
markdown = doc.export_to_markdown()
json_doc = doc.export_to_dict() # 含 bbox / page
html = doc.export_to_html()
text = doc.export_to_text()

# 重点 · 拿 chunk 级 metadata
for item in doc.iterate_items():
    if hasattr(item, "prov"):
        for p in item.prov:
            page = p.page_no; bbox = p.bbox # 用于 EvidenceAnchor
```

老司机说

5 个工程要点

- do_ocr=False · text-based 不必 OCR
- table cell 边界精确还原
- images_scale=2.0
- 图 2x 分辨率 · 后接 VLM caption
- iterate_items() 拿 prov
- page_no / bbox 直接喂 EvidenceAnchor
- accurate 模式
- 比 fast 慢 30% · 精度 +5%
- 财务 / 法律必上 accurate
- Granite-Docling 2025.9
- IBM Granite 模型增强
- 表格抽取业内 SOTA

TABLES · 比文本难 10 倍

表格处理 · 3 大难点 + 3 种存储策略

表格是 PDF 里硬骨头中的硬骨头 · 上线必专题处理

3 大难点

- Layout 识别(哪些是表)
- Cell 边界(合并 / 多行)
- 语义保留(纯 markdown 丢上下文)
- 5 列以上检索就掉精度
- 财报 / 价格表必专门处理

3 种存储策略

- A · 小表 (< 20 行) → markdown chunk
- B · 中表 → LLM 生成「表格描述」+ markdown 双轨
- C · 大表 → 转 SQL Table · NL2SQL Agent
- B 是 2026 主流默认 ·
- C 是财报场景必上

5 个抽取工具

- PDFPlumber · 开源简单表够
- Camelot · 开源精度高
- Tabula · Java 老牌
- LlamaParse · layout-aware 商业首选
- AWS Textract Tables · 最准最贵

```
# 策略 B · LLM 生成表格描述 + markdown 双轨
description = await llm.complete(f"基于这张 markdown 表格,用 2-3 句话描述: \n{md_table}")
chunk = {"content": md_table, "description": description, "table_id": uid,
        "schema": {"cols": header, "rows": len(rows)}} # 入库时双字段都进 embedding
```

INDUSTRY SIGNALS 2025

2024-2025 IDP 行业 4 个明显方向 · 趋势已收敛

不是「百家争鸣」 · 是「主干清晰 + 补丁式选型」

LlamaParse + Mistral OCR

2024-2026

LlamaIndex + Mistral 两条商业线 · 覆盖 To B 主战场。
2026 Mistral OCR 精度首次超 AWS Textract · 价格只一半。

Granite-Docling 2025.9

2025 末

IBM Docling + Granite 3.1B 视觉模型融合 · 完全开源。
表格抽取在 PubLayNet 上 SOTA · 中文友好。

ColPali / ColQwen2

2024.7-2025

彻底跳过解析 · 整页 PDF 当图像检索 · 视觉文档 RAG 范式革新。
2025 ColQwen2 出来中文也能用 · L05 详讲。

Anthropic Visual PDF

2025.3

Claude Sonnet 4.5 原生 PDF input · layout-aware 内建。
IDP 边界正在被 VLM 蚕食 · 传统流水线 + VLM 兜底是 2026 主流。

LESSON 02 · 语义切片

L02 从「按字数切」走到「按语义切」

加 2024 Late Chunking + Contextual Retrieval 两大游戏规则改变者

真实场景 · 90% 团队初版 RAG 长这样

我接过的项目 · 团队上来就 `text.split(每 512 字 chunk_size · overlap 50)` · Demo 跑通。
上线后客户问「合同第 5 条第 3 款怎么规定」—— 检索回来的 `chunk` 把 "第 5 条" 切到上一段尾巴, "第 3 款" 切到下一段开头, LLM 看到的是断裂的语义碎片。
那不是 `chunk` 算法笨 · 是「按字数切」根本不顾结构 · RAG 召回上限被切片就锁死了。

咱们这节聊 5 件事:

- 4 种主流切片策略 + Granularity 调参方法论
- Late Chunking (Jina 2024.09) + Contextual Retrieval (Anthropic 2024.09)
- 6 种特殊内容必须单独处理(表 / 代码 / 公式 / 引用 / 列表 / 对话)
- 代码 AST Chunking · TreeSitter · 不能按行切
- Voyage code-3 / BGE-Code · 代码 Embedding 选型

WHY CHUNKING MATTERS

切片决定 RAG 的召回上限 · 这话不夸张

Reranker 是「从候选里选最好」· 候选里没正确答案 · 神仙也无解

一个简单逻辑 · 绕不开

正确答案没被切到任何 chunk · ANN 检索召回 → Reranker 排不到 → LLM 看不到 → RAG 必错。
反过来 · 切片把上下文切碎 · 正确答案在 chunk 里但和上下文断裂 · LLM 看到孤立信息,信心低,出现幻觉或拒答。
所以切片不是「文字处理」· 是「语义对齐」· 这是 RAG 工程链路里 ROI 最高的一个动作。

切错的症状	根因	怎么修
答案漂 · 上下文断裂	按字符 / 按 token 暴力切	按结构 (heading / sentence) 切
财务数字串到错的表格	表格被 chunk 拦腰切	表格独立 chunk + 描述双轨
代码理解错 · 函数被切	按行切代码	TreeSitter AST 按函数切
多轮对话理解错	一问一答 chunk 分开	保留 turn 元信息或合并

4 STRATEGIES

4 种主流切片策略 · 按内容类型选

没有银弹 · 但有默认起点 —— 结构感知是 90% 场景的最优解

① Fixed-size

- 每 N 字符 / token 切
- + overlap 50-100
- 简单 / 通用 / 不顾结构
- 场景 · 纯散文 / 邮件
- 默认起点 · 但低质量

② Structure-aware ★

- 按 heading / section 切
- 保留 H1/H2 层级 metadata
- 90% 场景的最佳起点
- 场景 · PDF / Markdown / HTML
- IDP 输出后直接吃

③ Semantic

- LLM-based · 按主题切
- 语义连贯,但 cost 高
- embedding 相邻度切
- 场景 · 论文 / 长报告
- 5% query 走 Agentic

④ Late Chunking ★

- Jina 2024.09 创新
- 先 embed 整段,再 chunk
- 上下文不丢
- 场景 · 长文档
- 2026 检索 SOTA

GRANULARITY TRADE-OFF

切片粒度的权衡 · 没有最优,只有最适合

默认从 300-600 token 起步 · 按 retrieval@k 评测反推 · 不要凭感觉调

粒度	chunk_size (token)	优势	劣势	场景
超细	50-150	检索精度高 · 命中精确	LLM 看到孤立信息易幻觉	事实问答 / FAQ
细 ★	300-600	默认起点 · 平衡	复杂查询可能上下文不够	通用 RAG ★
中	600-1200	上下文充足 · 推理类好	检索精度略低	论文 / 长报告
粗	1200-2000	完整段落 · 推理理解强	召回不精确事实	法律 / 合同
超粗	> 2000	一整页 · 不损上下文	检索召回质量差	LongContext + RAG 混合

LATE CHUNKING

Late Chunking · Jina AI 2024.09 末游戏规则改变者

颠倒「先切再 embed」→「先 embed 再切」· 上下文不丢

原理 · 顺序颠倒 = 上下文保留

传统 · 先切 chunk → 每个 chunk 单独 embed → 每 chunk 失去整段上下文。

Late Chunking · 整段先过 encoder 拿 token-level embedding → 再按 chunk 边界对 token-embedding 池化(mean pool)。

效果 · 长文档检索 nDCG@10 平均 +9%(Jina paper) · 对「中间引用前文」类 chunk 修复显著。

```
# Late Chunking · Jina v3 / Embed v4 实现
from transformers import AutoModel
model = AutoModel.from_pretrained("jinaai/jina-embeddings-v3", trust_remote_code=True)

# 1. 整段过 encoder 拿 token-level embedding
inputs = tokenizer(long_text, return_tensors="pt", truncation=True, max_length=8192)
outputs = model(**inputs, output_hidden_states=True)
token_emb = outputs.last_hidden_state # [1, seq_len, dim]

# 2. 按 chunk 边界(预先用 structure-aware splitter 算)对 token-emb mean pool
chunk_embs = []
for (start, end) in chunk_boundaries:
    chunk_embs.append(token_emb[0, start:end, :].mean(dim=0))
```

老司机说

Late Chunking 适用

- 长文档 · > 2k tokens
- chunk 间互相引用强
- 论文 / 法律 / 技术文档

2026 实现

- Jina v3 原生支持
- OpenAI embedding 无法
- Voyage v3 也支持

ROI · 代码改动小 · 收益显著

- 不动切片机制
- 只换 embed 顺序
- nDCG +9%

CONTEXTUAL RETRIEVAL

Anthropic Contextual Retrieval · 35-49% 精度提升

不动切片机制 · 给每个 chunk 自动加「全文摘要前缀」

```
# Anthropic Contextual Retrieval · 2024.09 paper
import anthropic
client = anthropic.Anthropic()

CONTEXTUALIZE_PROMPT = """
<document>{doc}</document>

下面是上述文档中的一个 chunk:
<chunk>{chunk}</chunk>

请用一句话总结这个 chunk 在文档整体中的上下文 / 位置 /
所属节,以便检索时帮模型理解。只输出简短描述,不要复述 chunk 内容。
"""

def contextualize_chunk(doc: str, chunk: str) -> str:
    resp = client.messages.create(
        model="claude-haiku-4-5-20251001", # Haiku 跑省钱
        max_tokens=128,
        system=[{"type": "text", "text": CONTEXTUALIZE_PROMPT,
                  "cache_control": {"type": "ephemeral"}}], # 全文走 cache
        messages=[{"role": "user", "content":
                    CONTEXTUALIZE_PROMPT.format(doc=doc, chunk=chunk)}],
    )
    return resp.content[0].text

# 入库时把 context_prefix 拼到 chunk 前面再 embed
context = contextualize_chunk(full_doc, chunk)
augmented = f"{context}\n\n{chunk}"
embedding = embedder.embed(augmented)
```

老司机说

Anthropic paper 数据

- Recall@20 · -35% 错误率
- Contextual BM25 配合
- -49% 错误率

关键 · Prompt Caching

- 整篇文档进 cache · read 价 0.1x
- 一份长文档跑 200 chunk
- cache 命中后 cost 极低

2026 实战

- Haiku 4.5 · \$0.001-0.005 / chunk
- 整本 1000 chunk 书 \$5 以内

何时不上

- chunk 已经自带 H1/H2 上下文
- 短文档 (< 5k token)

SPECIAL CONTENT

6 种特殊内容必须单独处理 · 套通用规则一定坏

通用切片器在 Demo 阶段够用 · 生产里至少要 6 个分支

类型	错误做法	正确做法	工具
表格	混进文本流 cut	独立 chunk + 描述双轨	LlamaParse / Camelot
代码	按行切	AST 按函数切	TreeSitter ★
公式	抽成乱码	保 LaTeX / MathML	Mathpix / Marker
列表 / 枚举	断在 item 中间	整列表当原子 chunk	Marker 自带
对话 / FAQ	问答分开	turn-aware 保 Q+A 一组	自定义
引用 / 脚注	混入正文	入 metadata,不混正文	GROBID / Marker

CODE AST CHUNKING

代码不能按行切 · 必须按 AST 结构切

```
# pip install tree-sitter tree-sitter-languages
import tree_sitter_languages
from tree_sitter import Parser
```

```
# Python · Java / Go / TS / Rust 同接口
parser = Parser()
parser.set_language(tree_sitter_languages.get_language("python"))
```

```
def chunk_by_ast(code: str, max_size: int = 1500) -> list[dict]:
    tree = parser.parse(code.encode())
    chunks = []
```

```
def walk(node, parent_ctx=""):
    # 函数 / 方法 · 独立 chunk
    if node.type in ("function_definition", "class_definition"):
        content = code[node.start_byte:node.end_byte]
        name = next((c.text.decode() for c in node.children
                     if c.type == "identifier"), "?")

        if len(content) <= max_size:
            chunks.append({
                "type": node.type, "name": name,
                "content": parent_ctx + content, # 含 import 上下文
                "start_line": node.start_point[0],
                "end_line": node.end_point[0],
            })
        else:
            # 大类拆方法
            for c in node.children:
                walk(c, parent_ctx=content[:200] + "...\\n\\n")
    for c in node.children:
        walk(c, parent_ctx)
```

```
walk(tree.root_node)
return chunks
```

老司机说

4 种切法

- 按函数 · 每函数一 chunk
- 按类 · 大类拆方法
- 按文件 · 小文件整切
- 保 imports + class 上下文 = anchor

TreeSitter 优势

- 50+ 语言一统 · Python/Go/Java/TS
- 比 ast 模块通用 · 跨语言
- GitHub Code Search 同款引擎

2026 共识

- 代码 chunking 必须 AST
- 单 chunk 上限 1500-2000 token
- 函数 > max_size 时拆方法
- 永远保 imports + 注释



CODE EMBEDDING

代码 Embedding 选型 · 通用 embedding 不灵

代码用 text embedding 在 CodeSearchNet 上掉 20% NDCG · 必须专 model

模型	类型	关键特性 2026	价格	适用
Voyage code-3	商业 SaaS	代码场景 SOTA · CodeSearchNet 比 OpenAI +22%	\$0.06/1M tok	代码 RAG 首选 ★
Jina-embeddings-v3 code	商业 + 开源	Late Chunking + 代码场景双优	\$0.02/1M tok	通用 + 代码兼顾
BGE-Code-v1	开源 (BAAI)	2025 最强中文代码 embedding	\$0	中文项目 ★
CodeBERT / GraphCodeBERT	开源 (微软)	学术经典 · 老牌	\$0	研究用途
OpenAI text-embed-3	商业 SaaS	通用强 · 代码尚可	\$0.13/1M tok	通用 fallback
SantaCoder embedding	开源	小模型 110M · 推理快	\$0	边缘部署

REAL ARTIFACT

结构感知切片实现 · OmniSupport 项目

Marker 输出 markdown → 按 heading + 长度双约束切

```
# pipelines/chunker/structure_aware.py
from typing import Iterator
from langchain_text_splitters import MarkdownHeaderTextSplitter

def structure_aware_chunks(md: str, chunk_size: int = 500,
                           overlap: int = 50) -> Iterator[dict]:
    # Phase 1 · 按 heading 切大段
    header_splitter = MarkdownHeaderTextSplitter(
        headers_to_split_on=[("#", "H1"), ("##", "H2"), ("###", "H3")],
    )
    sections = header_splitter.split_text(md)

    for sec in sections:
        text = sec.page_content
        meta = sec.metadata # {"H1": "...", "H2": "..."}

        # Phase 2 · 大段内按句子 + 长度切小 chunk
        if len(text) <= chunk_size:
            yield {"content": text, "metadata": meta}
        else:
            sents = sentence_split(text) # 用 PySBD / spaCy
            buf, buf_len = [], 0
            for sent in sents:
                if buf_len + len(sent) > chunk_size and buf:
                    yield {"content": " ".join(buf), "metadata": meta}
                    buf = buf[-overlap:] # 滑窗 overlap
                    buf_len = sum(len(s) for s in buf)
                buf.append(sent); buf_len += len(sent)
            if buf:
                yield {"content": " ".join(buf), "metadata": meta}
```

老司机说

4 个工程要点

- 2 phase 切 · 先 heading 后句子
- heading 入 metadata = chunk 上下文
- overlap 50 token 滑窗
- 防止句子被切碎
- sentence_split 用 PySBD
- 比 nltk 准确 · 中英双语
- chunk metadata 含
- H1/H2/H3 标题层级
- source / page / bbox
- 后续 EvidenceAnchor 直接用

CHUNKING ECOSYSTEM 2025

切片生态 · 4 个值得关注的实现

2025 共识 · 切片是「检索质量 + 业务结构 + token 经济」三方权衡

LangChain Splitters

2024-2026

Markdown / HTML / Recursive Char / Code · 主流 + 开源 + 文档全。
90% Python 项目用它起步 · 跟 LangGraph 集成天然。

LlamaIndex NodeParser

2024-2026

Sentence / Semantic / Hierarchy / Window 4 种 · Semantic 走 LLM 切。
跟 LlamaIndex 检索 stack 无缝接 · Agentic RAG 友好。

Late Chunking · Jina

2024.09

Embedding 顺序颠倒 · 长文档检索 nDCG +9%。
不动切片机制 · 改 embed pipeline 即可 · ROI 高。

Contextual Retrieval · Anthropic

2024.09

Chunk 自动加全文摘要 prefix · Recall 错误率 -35-49%。
Haiku 4.5 + Prompt Caching = 整本书 \$5 以内。



HOW TO TUNE

怎么调切片参数 · 看评测指标,不是凭感觉

指标	指标作用	调参方向	阈值
Recall@5	召回:前 5 个含正答比例	太低 → 加 chunk_size / 加 overlap	> 0.80
Recall@20	召回兜底	太低 → 换 embedding / 加 hybrid BM25	> 0.92
MRR	正答平均排名	低 → 上 reranker	> 0.65
Faithfulness	答案是否扎根 chunk	低 → 增大 chunk size / 加 ctx	> 0.85
Context Precision	上下文相关度	低 → 减 chunk size / 加 rerank	> 0.80
Latency	检索 + 生成总耗时	高 → 减 top-k / 加 cache	P95 < 3s

老司机说 调参方法论 · 先 default → eval 跑 200 sample → 看哪个指标失守 → 反推调哪个参数 · 不要瞎试

LESSON 03 · 证据链

L03 客户对 RAG 系统最常见的两个质疑

"这答案哪来的" / "让我看原文" · 生产 RAG 必答

真实场景 · 金融客户的 demo 评审现场

我去年带团队给一家头部券商做 RAG Demo · PMF 都过了 · 客户技术评审会上突然问:「这个答案是来自哪份研报?能不能给我跳到那一页?」
团队当场卡 15 秒 · 因为 chunk 入库时把 page_no / bbox 都丢了 · 只剩纯文字 · 根本回不到原文。
那一刻评审没过 · 6 个月的功夫归零 · 根因是「证据链」从一开始就没设计。

咱们这节聊 5 件事:

- 5 levels of evidence · 你的 RAG 在哪一档
- EvidenceAnchor schema · Identity / Location / Context 必含 3 组字段
- Anthropic Citations API 2025.1 GA · 字符级定位 · 幻觉率 -50%
- 5 层端到端无损传递 · 90% 团队在 Index → Retrieve 这步丢字段
- 审计 + Governance · Week 14 治理周的入口



4 LEVELS OF EVIDENCE

证据链 5 个等级 · 你的 RAG 在哪一档

生产级及格线 = L3 段落 + 坐标 · 低于 L3 客户对系统建立不起长期信任

等级	能力	颗粒度	场景	客户接受度
L0	只给答案 · 无 source	无	Demo only	低 · 必拒
L1	给文档名 (source field)	文档级	内部使用	低
L2	页码 + section 标题	段落级	内部知识库	中
L3 ★	页码 + bbox + 段落引用	坐标级	To B 生产线	高 ★ 及格线
L4 ★★	字符级 + Citations API	token 级	金融 / 法律 / 医疗	极高 · 合规必备



EVIDENCEANCHOR FIELDS

EvidenceAnchor schema · 必含 3 组字段

Identity 能回原资料 · Location 能定位 · Context 能解释

Identity · 身份

- doc_id · 原文档主键
- version_id · 避免文档更新丢链
- chunk_id · 本 chunk 唯一 id
- source · e.g. "财报2025_Q3.pdf"
- 不全 → 客户问追溯不到

Location · 定位

- page_no · 页码
- bbox · [x0,y0,x1,y1] 坐标
- span_start · 字符级偏移
- span_end · 字符级偏移
- 不全 → 前端没法高亮

Context · 解释

- heading_path · H1 > H2 > H3
- rerank_score · 排序证据
- retrieval_method · hybrid / vector
- release_id · 哪次发布命中
- 不全 → 审计断链

REAL ARTIFACT · EVIDENCEANCHOR

EvidenceAnchor 真实数据结构 · OmniSupport 项目

前端拿到这个结构 · 一键高亮原 PDF + 显示置信度热力图

```
# services/rag_api/models.py · Pydantic Schema
from pydantic import BaseModel, Field
from typing import Literal

class EvidenceAnchor(BaseModel):
    # === Identity · 必填 ===
    doc_id: str = Field(..., description="文档主键")
    version_id: str = Field(..., description="文档版本,避免更新断链")
    chunk_id: str = Field(..., description="chunk 唯一 id")
    source: str = Field(..., description='e.g. "财报2025_Q3.pdf"')
    source_type: Literal["pdf", "html", "audio", "video", "code"] = "pdf"

    # === Location · 必填 ===
    page_no: int = Field(..., ge=1, description="页码 1-based")
    bbox: list[float] = Field(..., min_items=4, max_items=4) # [x0,y0,x1,y1]
    span_start: int = Field(..., description="字符级起始")
    span_end: int = Field(..., description="字符级结束")

    # === Context · 必填 ===
    heading_path: list[str] = Field(default_factory=list) # ["H1", "H2", "H3"]
    rerank_score: float = Field(..., ge=0.0, le=1.0)
    retrieval_method: Literal["hybrid", "vector", "bm25", "graph"] = "hybrid"
    release_id: str = Field(..., description="哪次 release 命中")

    # === 内容 ===
    chunk_text: str
    confidence: float = Field(..., ge=0.0, le=1.0)
```

ANTHROPIC CITATIONS API

模型自己输出引用片段 · 不再靠 *prompt hack* 让 LLM 说出来源

```
# Anthropic Citations API · 2025.1 GA
import anthropic
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-5-20251022",
    max_tokens=1024,
    messages=[
        {
            "role": "user",
            "content": [
                # 关键 · documents 块 · citations.enabled=True
                {
                    "type": "document",
                    "source": {"type": "text", "media_type": "text/plain",
                               "data": chunk_1.text},
                    "title": chunk_1.source,
                    "context": chunk_1.heading_path,
                    "citations": {"enabled": True},
                },
                {
                    "type": "document",
                    "source": {"type": "text", "data": chunk_2.text},
                    "title": chunk_2.source,
                    "citations": {"enabled": True},
                },
            ],
            {"type": "text", "text": "请基于上述文档回答 ..."},
        }
    ],
)

# 答案里每一句话自动带 cited_text 段
for block in response.content:
    if block.type == "text":
        for citation in block.citations or []:
            print(citation.cited_text, citation.document_index)
```

老司机说

关键能力

- 模型字符级输出 citation
- cited_text + document_index
- 不需要 prompt hack

生产数据 · Anthropic 公开

- 幻觉率 -50%
- 答案准确率 +18%
- 不增加延迟

2026 共识

- To B 必上 Citations API
- OpenAI 也跟进
- Structured Outputs + tool_use

配套

- prompt cache 缓存 system
- document 每篇 entry 单独标
- max_tokens 不必加 · 开销持平

END-TO-END FLOW

证据链 5 层端到端无损传递 · 任何一层丢字段下游全断

90% 团队在 Index → Retrieve 这步丢字段 · 为优化向量库性能把业务 metadata 丢了

Parse	Chunk	Index	Retrieve	Generate
抽取 + 坐标 doc_id / page_no / bbox / span 必入	切片 + 元数据 heading_path / version / chunk_id 入 metadata	向量化 + ANN metadata 跟 vector 绑定 · 别砍	混检 + 排序 rerank_score / retrieval_method 标	LLM + 引用 release_id / cited_text 回写 evidence

踩坑提醒

断链高发地 · Index 阶段为了 ANN 性能砍 metadata · 上线后客户问追溯,只能空白响应。

2026 共识 · metadata 必跟 vector 同库存 · pgvector / Milvus 2.5+ 都支持 hybrid metadata · 不要为省 10% 内存丢业务字段

ANTI-PATTERNS

证据链 4 个反模式

存了不用 / 用了不对 / 评测不验证 · 这是三大主流坑

反模式	具体表现	后果	正确做法
没存 bbox	chunk 只有文本 + page	前端不能高亮原文	Parse 阶段必入 bbox
存了不传	API 返回只给答案不给 evidence	客户问追溯空白	RAGAnswer 必带 evidence[]
传了不对齐	答案 cite 第 1 chunk 但实际看的第 3	evidence 假 · 比没有更危险	Citations API 强对齐
评测不验证	Eval 只看答案不看 citation	上线后 30% citation 是错的	Ragas citation_accuracy 必测

踩坑提醒

我跟你讲那个金融评审的结局 · 团队回去花 3 个月重做 evidence pipeline · Parse 阶段重新跑全量 PDF + 重建索引。客户后来签了 · 但 6 个月信任损耗 · 这种代价比技术成本高 10 倍 · 从 day 1 就要设计 evidence



INDUSTRY SIGNALS 2025

证据链已是 To B 合规基线 · 大厂都在做

2025 起证据链不是「锦上添花」——是 AI 产品对 B 端的合规基线

Anthropic Citations API

2025.1 GA

字符级 cited_text · 幻觉率 -50% · 模型原生输出引用。
Claude Sonnet 4.5 / Opus 4.5 / Haiku 4.5 全线支持。

OpenAI Structured Outputs

2024.8 + 2025 强化

Pydantic schema 直传 · citation 字段强约束。
GPT-4o / GPT-5 都支持 · 0 schema 异常。

Perplexity Sonar

2024-2025

答案带 footnote 形式引用 · 对话型 RAG 标杆。
C 端 Citations 体验的 reference 实现。

EU AI Act 合规

2024.8 生效

高风险 AI 强制 traceability · Citations 是合规基础。
2026 中欧合规审查必看 · 是 To B 上线刚需。



AUDIT + GOVERNANCE

证据链支撑全链路审计 · Week 14 治理的入口

证据链做对了 · 合规审计从「开会扯皮」变成「点几下出报告」

```
# governance/audit_query.py · 客户问「3 个月前那次回答凭啥」
SELECT
  answer.text,
  evidence.doc_id, evidence.version_id, evidence.page_no, evidence.bbox,
  evidence.chunk_text,
  answer.release_id,          -- 哪次 release 产出
  rel.prompt_version, rel.model_version, rel.index_snapshot -- L05 原子绑定回查
FROM rag_answer_log answer
JOIN evidence ON answer.id = evidence.answer_id
JOIN releases rel ON answer.release_id = rel.release_id
WHERE answer.user_id = ? AND answer.created_at BETWEEN '2026-02-01' AND '2026-02-15'
ORDER BY answer.created_at;
```

-- 这一条 SQL 拉出完整链路 · 审计员从 answer 反推到 doc 哪页 / 哪个 chunk / 哪版 prompt / 哪版索引

老司机说

审计场景 4 类 · 日常都用证据链支撑

- 客户投诉 · 「答案错了」 → 反查 chunk + version 看哪一步出问题 → 入 bad case
- 监管检查 · 「3 个月前你怎么答的」 → release_id 锁定 + evidence 还原
- 内部 review · 「为什么改了答案」 → 对比新旧 release 的 prompt/model/index 差异
- 合规年审 · EU AI Act 要 traceability · 这一条 SQL 出 90% 报告

LESSON 04 · 切片质检

L04 50-200 份样本 + 反例库 + CI 门禁

专业团队的标准操作 · 不是「上线后再说」

真实场景 · 「先上线试试」的真实代价

前年我做过一个客户案例 · 团队抢工期 · 切片没做质检直接上线 · 上线 3 天客户反馈「答案乱说话」。
排查后发现 · Marker 抽 PDF 时 5% 的页面 layout 解析错(多栏识别失败) · 下游 chunk 噪声严重 · 幻觉率 18%。
回滚 + 重新切 + 重建索引 · 停服 36 小时 · 外加客户信任损耗。如果上线前跑 100 sample 抽样 · 这种事故根本不会发生。

咱们这节聊 5 件事：

- 质检 4 大维度 · 完整性 / 噪声 / 证据 / 连贯 · 每维都有指标 + 阈值
- 70% 分层 + 20% 困难 + 10% 对抗 · 抽样策略的最优组合
- 6 个量化指标 + CI 门禁 · gate_decision = block 时阻断索引构建
- Drift Detection 3 监控点 · 上线后情况会变 · 早发现早修
- PDF 增量更新策略 · diff → byte range → chunk 级更新

QUALITY AS GATE

质检不是「看几个 chunk」· 是 CI 门禁的工程流程

BI 时代的 `not_null` / `unique` 在这里不够用

维度	BI 时代质检	RAG 时代质检	差异
对象	表格 / 行 / 列	chunk / 文档 / 多模态	颗粒不同
指标	<code>not_null</code> / <code>unique</code> / <code>range</code>	完整性 / 噪声 / 证据 / 连贯	语义化
工具	GreatExpectations / Soda	Ragas / DeepEval / Phoenix	专用
评测者	人工 + 规则	LLM-as-Judge + 规则	半自动化
抽样	全表跑 / 抽样跑	分层 + 困难 + 对抗	更聪明
门禁	block 写入	<code>gate_decision</code> = block 索引	同思路
频率	每次写	每次切 + 每天 drift	加频

4 QUALITY DIMENSIONS

切片质检 4 大维度 · 少一个就有盲区

完整性 / 噪声 / 证据 / 连贯 · 每个都有指标 + 阈值 + 不达标根因

① 完整性

- chunk 覆盖率 · 100% 内容入库
- 关键 entity 召回率
- 不达标 → 切片粒度太大 / split 错
- 指标 · coverage > 95%
- 工具 · 自研 set diff

② 噪声

- 页眉页脚 / 水印 / nav 混入
- 空 chunk / 重复 chunk
- 不达标 → IDP layout 还原弱
- 指标 · noise_ratio < 3%
- 工具 · 规则 + LLM 抽检

③ 证据

- EvidenceAnchor 必字段完整
- bbox / page_no / version_id 不空
- 不达标 → Parse 阶段砍字段
- 指标 · evidence_complete = 100%
- 工具 · Pydantic 强校验

④ 连贯

- chunk 句子结构完整
- 不切在句中 / 段中
- 不达标 → splitter 算法过粗
- 指标 · sentence_break_rate < 1%
- 工具 · PySBD / LLM 抽检

SAMPLING STRATEGIES

抽样策略 · 纯随机是错误起点

生产级 = 70% 分层 + 20% 困难 + 10% 对抗 · 这是平衡覆盖和聚焦的最优组合

抽样类型	占比	怎么选	目的
分层抽样	70%	按 doc_type (财报/合同/手册) + 长度 + 章节均匀采样	保证覆盖代表性
困难样本	20%	历史 bad case + LLM-judge 低分 + 复杂 layout 页	聚焦关键风险
对抗样本	10%	边界 case + 极长 / 极短 / 极复杂表格	验证 robustness
纯随机(错误)	0%	从全集随机抽 100 个	掩盖关键问题
抽样规模	50-500	小项目 50 · 生产 200 · 关键场景 500	准确性 + 成本平衡

老司机说

抽样策略 ROI 数据 · 我做过的 3 个 RAG 项目对比
· 纯随机 100 sample · 漏检率 25%
· 70/20/10 分层 200 sample · 漏检率 4%
· 多花的 100 样本标注成本 · \$50 · 避免上线事故损失 \$50K · ROI > 1000x

REGRESSION COMPARISON

版本对比 · 策略迭代不再像赌博

没有版本对比 · 改切片策略每次都是「拍脑袋」 · 专业团队 4 种比法配合用

A · 指标对比

- 新旧版本同一 eval set
- Recall@5 / MRR / Faith 对比
- 阈值 · 不能比旧版退步 3%
- 工具 · Ragas + pandas
- 场景 · PR merge 强制

B · 答案 diff

- 同一 query 跑新旧版
- 答案文字 diff
- LLM-as-judge 评哪个对
- 工具 · Phoenix / 自研
- 场景 · prompt 大改

C · Bad case 回归

- 上线后积累的 bad case
- 每次大改都跑
- 阈值 · 老的 bad case 都能修
- 工具 · 自研 jsonl
- 场景 · 迭代必跑

D · 流量 A/B

- 5% canary · 对比指标
- CSAT / Citation Acc 监控
- 阈值 · 退化 → 自动回滚
- 工具 · LaunchDarkly
- 场景 · 上线必走

REAL ARTIFACT · QUALITY REPORT

切片质检报告 schema · OmniSupport 项目

CI 阻断的核心 · *gate_decision = block* 就停止索引构建

```
# pipelines/quality/report.py · Pydantic Quality Report
from pydantic import BaseModel, Field
from typing import Literal

class QualityReport(BaseModel):
    run_id: str
    doc_set_id: str
    sample_size: int = Field(..., ge=50, le=2000)
    sampling: dict = Field(..., description='{stratified: 0.7, hard: 0.2, adversarial: 0.1}')

    # 4 大维度 量化指标
    completeness_coverage: float = Field(..., ge=0.0, le=1.0) # 阈值 > 0.95
    noise_ratio: float = Field(..., ge=0.0, le=1.0) # 阈值 < 0.03
    evidence_complete_rate: float = Field(..., ge=0.0, le=1.0) # 阈值 = 1.0
    sentence_break_rate: float = Field(..., ge=0.0, le=1.0) # 阈值 < 0.01

    # 检索质量
    recall_at_5: float
    recall_at_20: float
    mrr: float
    faithfulness_llm_judge: float # LLM-as-Judge

    # 门禁决策 · CI 拿这个字段做决定
    gate_decision: Literal["pass", "warn", "block"]
    block_reasons: list[str] = Field(default_factory=list)

    # Drift 监控
    drift_metrics: dict = Field(default_factory=dict)
    embedding_drift_kl: float = Field(default=0.0)
```




6 QUANTITATIVE METRICS

切片质量 6 个量化指标 · 少一个就有盲区

这是切片层的「生产准入条件」——跟 SLA 一样必须签

指标	定义	阈值 (生产)	不达标根因	修复方向
Coverage	内容入库覆盖率	> 95%	split 漏 / 大段没切	检查 chunker 边界
Noise Ratio	噪声 chunk 占比	< 3%	IDP layout 还原弱	换 Marker / 加 VLM 兜底
Evidence Complete	metadata 完整率	= 100%	Parse 阶段砍字段	强 Pydantic 校验
Sentence Break	句子被切碎率	< 1%	splitter 算法过粗	用 PySBD 切
Recall@5	检索召回@5	> 80%	chunk_size 不对 / embedding 弱	调粒度 / 换 embedder
Faithfulness	答案扎根 chunk 度	> 85%	chunk 上下文不够	加 chunk_size / Late Chunking

EVAL ECOSYSTEM 2025

评测生态 · 4 个值得用的工具

不用造轮子 · 「切片质量」和「模型质量」被同等对待

Ragas

2024-2026 开源

Faithfulness / Context Precision / Recall / Citation Acc 一体。
Python 友好 · LangChain 集成 · 2026 业内事实标准。

DeepEval

2024-2026 开源

pytest 风格 · CI 友好 · 30+ 指标内置。
GitHub 7k stars · 适合工程团队 PR 阻断。

Arize Phoenix

2024-2026 开源

Eval + Trace + Prompt Registry 一体。
自托管首选 · 2025 加 Pretrained Evaluators。

Langfuse

2024-2026 开源 + 商业

observability + eval + dataset 三件套。
2026 加 LLM-as-Judge 工作流 · SOC2 合规友好。

QUALITY GATE FLOW

切片质量门禁 · 从生成到上线 5 个卡点

质检接进 CI/CD · 切片质检不过 · 索引构建不允许启动

① 切片完成	② 抽样 + 标注	③ Ragas 跑评测	④ CI 门禁判断	⑤ 索引构建
产物入暂存 chunks.jsonl 暂存目录	70/20/10 抽样 200 sample 人工 30 min	6 指标 + LLM-Judge quality_report.json gate_decision	block / pass / warn gate=block → PR 阻断	通过后启动 index_v2 ← release 绑定

老司机说

5 个卡点是「不可分」· 漏一个就会有「不达标切片」混进索引。
我跟你讲那个 36 小时事故· 根因就是漏了步骤 ③ (没跑评测)· 直接从切片到索引。后来我们把这 5 步硬编码到 GitHub Actions · pre-merge gate · 从此再没翻车

DRIFT DETECTION

数据漂移监测 · 上线后情况会变 · 3 个监控点

老司机讲 · 上线 1-3 个月效果一定会漂 · 早发现早修

① Chunk 数量漂

- 新文档进库 vs 老文档比例
- 统计 · 新增 chunk 数周环比
- 阈值 · > 50% 突变 → 告警
- 修法 · 全量重切 + Ragas 复评
- Alibi · Evidently AI

② Embedding 分布漂

- 新数据 vs 老索引向量分布
- 统计 · KL divergence / MMD
- 阈值 · > 0.15 → 告警
- 修法 · re-embed 增量数据
- Alibi · Evidently / Phoenix

③ Answer 质量漂

- Faithfulness / CSAT 周环比
- 统计 · Faith < 0.85 → 告警
- 原因 · 模型升级 / prompt 退步
- 修法 · 回滚 release_id
- Alibi · Ragas + Slack hook

INCREMENTAL UPDATE

PDF 增量更新 · diff → byte range → chunk 级更新

文档改一行不能全量重切 · chunk 级精准更新省 90% 算力

```
# pipelines/incremental/update.py · PDF 增量更新
import hashlib
from pathlib import Path

def incremental_update(new_pdf: Path, old_version_id: str):
    """文档改了 → diff → 受影响 chunk → 只重切 / 重 embed 这些"""

    old_chunks = chunk_store.load(old_version_id)
    new_md = marker_parse(new_pdf)          # 重新 parse
    new_chunks_full = structure_aware_chunks(new_md)

    # 1. 算每个 chunk 的 content hash
    def chunk_hash(c): return hashlib.md5(c["content"].encode()).hexdigest()
    old_hashes = {chunk_hash(c): c for c in old_chunks}
    new_hashes = {chunk_hash(c): c for c in new_chunks_full}

    # 2. diff · 找新增 / 删除 / 不变
    added = [new_hashes[h] for h in (set(new_hashes) - set(old_hashes))]
    removed = [old_hashes[h] for h in (set(old_hashes) - set(new_hashes))]
    kept = set(new_hashes) & set(old_hashes)

    # 3. 只对增量做 embed + index
    new_embeddings = embedder.embed_batch([c["content"] for c in added])
    index.insert(added, new_embeddings)
    index.delete([c["chunk_id"] for c in removed])

    # 4. 记录 release_id 绑定
    new_version_id = f"v{int(old_version_id.lstrip('v'))+1}"
    return {
        "added": len(added), "removed": len(removed),
        "kept": len(kept), "version_id": new_version_id,
    }
```

老司机说

增量更新 ROI

- 全量重切 1000 页 PDF
- · embedding cost: \$5
- · 耗时: 15 min
-
- 增量更新只重切 50 chunk
- · embedding cost: \$0.5
- · 耗时: 30 sec
-
- 节省 90% 算力 + 96% 时间

关键设计

- content hash 是主键
- 用 md5 / sha256
- hash 不变 → chunk 不变
- 不依赖 byte range 易碎

版本绑定

- version_id 跟 release.yaml 绑
- 客户问追溯有据可查

LESSON 05 · 多模态扩展

L05 不重新发明轮子

Parse / Chunk / Evidence / Quality 范式 · 直接换轨用于音视频

真实场景 · 「我们要做 Video Understanding!」

客户找我做项目 · 一开口就是「我要 video understanding · 我要 AI 看视频回答」 · 团队傻眼 · 以为要从零造一套新管道。
实际呢 —— 音频 / 视频 / 图 · 本质就是「另一种非结构化数据」 · L01-L04 那套 (Parse / Chunk / Evidence / Quality) 完全可以套用 · 只是物理载体换了。
关键是「同构思维」 · 音频有 segment 就像 PDF 有 page · 视频有时间戳就像合同有 bbox · 别被「多模态」三个字唬到。

咱们这节聊 6 件事：

- Vision LLM 时代 · GPT-5V / Claude Sonnet / Gemini 2.5 / Qwen-VL 横评
- Whisper-Large-v3-Turbo + Pyannote Diarization · 完整音频处理代码
- Gemini 2.5 原生视频输入 + 关键帧 + Caption 双轨架构
- CLIP-style Embedding · OpenAI CLIP / SigLIP / Jina-CLIP / BGE-VL
- Multimodal RAG 3 架构(全转文本 / 多 store / Unified) · Hybrid 推荐
- ColPali / ColQwen2 · 视觉文档检索新范式

THREE-TRACK MODEL

音视频「三轨道」模型 · 工程化处理的统一抽象

Audio / Vision / Time-Stamp 三条轨独立处理 + 时间戳对齐

Audio Track · 音轨

- ASR 转写 · Whisper-v3-Turbo
- Diarization · pyannote 3.1
- 说话人 + 时间戳 + 文字
- 入 chunk 带 speaker_id
- 工业默认

Vision Track · 视频轨

- 关键帧抽取 · ffmpeg / OpenCV
- VLM Caption · Claude / Gemini
- 每 N 秒 1 帧 / 场景切换检测
- 入 chunk 带 frame_path
- 可视化检索

Time-Stamp · 对齐轨

- segment_ts · 音 + 视 + 文统一
- 同一秒发生的内容互相关联
- evidence 跳转到精确时刻
- 入 chunk 带 segment_ts
- 同构 EvidenceAnchor

AUDIO VS DOCUMENT

音频处理 vs 文档处理 · 看似不同 · 本质同构

5 个维度对照 · 音频不是新数据类型 · 是同一套范式的物理形态

维度	文档 (PDF)	音频 (mp3)	同构关系
Parse	Marker → markdown	Whisper → 文字 + 时间戳	都抽语义
Chunk	按 heading / 句子	按 speaker / 话题	都按结构
Evidence	page_no + bbox	segment_ts + speaker_id	都可定位
Quality	4 维质检 / Coverage	4 维质检 / WER + Diarization Acc	都可量化
Index	pgvector 入	pgvector 入 + audio_url metadata	同 store
Retrieve	Hybrid + Rerank	Hybrid + Rerank	同链路
前端	高亮原 PDF	跳转音频时刻	统一 UX

VIDEO PIPELINE

视频处理流水线 · 三轨分头处理 + 时间戳对齐

视频比音频复杂 · 但工程化逻辑还是一样 · 关键是 `segment_ts` 把三轨对齐起来

① Demux	② ASR	③ Diarization	④ KeyFrame	⑤ VLM Caption	⑥ Align
ffmpeg 拆音 + 视 audio.wav frames/*.jpg	Whisper-v3-Turbo 转写 + 字级 时间戳	pyannote 3.1 speaker A/B/C 时间段	scene-detect 切 每 N 秒 1 帧 场景变化	Claude / Gemini 每帧 caption + OCR	segment_ts 合 三轨同 ts 同 chunk

老司机说

关键设计 · `segment_ts` 是「主键」 · 把音 / 视 / 文三个独立处理的结果黏合

实战 · 1 小时视频 · ASR ~10 min · KeyFrame 抽 600 帧 · VLM Caption ~5 min · 全 pipeline 总 < 25 min · 成本 < \$8

REAL ARTIFACT · AUDIO SEGMENT

音频 segment 真实数据结构 · OmniSupport 项目

schema 和文档 EvidenceAnchor 完全同构 · 这就是「同一套范式」的力量

```
# services/audio_pipeline/models.py
from pydantic import BaseModel, Field
from typing import Literal

class AudioSegment(BaseModel):
    # === Identity (同 EvidenceAnchor) ===
    doc_id: str = Field(description="# 音频文件主键")
    version_id: str
    chunk_id: str
    source: str = Field(description="# 'interview_2026_05.mp3'")
    source_type: Literal["pdf", "audio", "video", "image", "code"] = "audio"

    # === Location · 音频用 segment_ts 替代 page+bbox ===
    segment_ts_start: float = Field(..., description="秒级开始")
    segment_ts_end: float = Field(..., description="秒级结束")
    audio_url: str = Field(..., description="原音频 URL")

    # === Audio-specific ===
    speaker_id: str = Field(..., description="说话人 A/B/C")
    speaker_label: str = Field(default="", description="可选 · 真实姓名")
    confidence_asr: float = Field(..., ge=0.0, le=1.0)
    confidence_diarization: float = Field(..., ge=0.0, le=1.0)
    language: str = "zh"

    # === Context (同 EvidenceAnchor) ===
    rerank_score: float = Field(..., ge=0.0, le=1.0)
    retrieval_method: Literal["hybrid", "vector", "bm25"] = "hybrid"
    release_id: str
    chunk_text: str = Field(description="# ASR 文字")
    heading_path: list[str] = Field(default_factory=list, description="# 可选话题层级")
```

VISION LLM 2026

Vision LLM 时代 · 2024 前是 OCR + 分类各种 specialized model · 2026 一个 VLM 都能干

老司机讲 · 90% 图像任务现在用一个 VLM 调 API 即可 · 不必再训 specialized 模型

模型	类型	关键特性 2026	价格	适用
GPT-5 Vision	商业 SaaS	通用最强 · 复杂图表 · 长文档	\$0.005/img	通用 first choice
Claude Sonnet 4.5	商业 SaaS	多模态 + 200k context · 文档 OCR 极强	\$3/M tok	文档 / 截图 ★
Claude Opus 4.5	商业 SaaS	顶级理解 · 复杂图表分析	\$15/M tok	高价值场景
Gemini 2.5 Pro	商业 SaaS	2M context · 原生视频输入 · 多模态最广	\$1.25/M tok	视频 / 长上下文 ★
Qwen2.5-VL 72B	开源 (阿里)	2025 中文 VLM 最强 · 商业可用	\$0 (自托管)	中文 / 自托管 ★
InternVL 2.5	开源 (上海 AI Lab)	精度接近 GPT-4V · 多语言 · 学术活跃	\$0	研究 + 开源
Llama 3.2 Vision	开源 (Meta)	11B / 90B · 美企开源主流	\$0	英文 / 通用开源

REAL ARTIFACT · WHISPER + DIARIZATION

Whisper-Large-v3-Turbo + pyannote 3.1 · 完整音频处理代码

OpenAI 2024.10 出 Turbo · 比 v3 快 8x · 精度只降 1% · 生产首选

```
# pipelines/audio/process.py · Whisper Turbo + Diarization 完整代码
import whisper
from pyannote.audio import Pipeline
from pydub import AudioSegment as PydubSegment

# 1. ASR · Whisper-Large-v3-Turbo (2024.10 · 比 v3 快 8x)
asr = whisper.load_model("large-v3-turbo")
result = asr.transcribe(
    "interview.mp3", language="zh",
    word_timestamps=True, condition_on_previous_text=False,
)
# result["segments"] · 含每段 + 字级时间戳

# 2. Diarization · pyannote 3.1
diar = Pipeline.from_pretrained("pyannote/speaker-diarization-3.1",
                                use_auth_token="hf_xxx")
diarization = diar("interview.wav")
# 输出 [(start, end, speaker_label), ...]

# 3. 合并 · ASR 文字段 + speaker label
def merge(asr_segs, diar_segs):
    merged = []
    for seg in asr_segs:
        mid = (seg["start"] + seg["end"]) / 2
        # 找 mid 落在哪个 diar segment 里
        spk = next((s for s in diar_segs if s.start <= mid <= s.end), None)
        merged.append({
            "start": seg["start"], "end": seg["end"],
            "text": seg["text"], "speaker": spk.label if spk else "?",
            "words": seg["words"],
        })
    return merged
```

GEMINI 2.5 NATIVE VIDEO

Gemini 2.5 原生视频输入 · 直接看视频回答

2025 主流 VLM 都在跟进 · 跳过音视频拆分流水线 · 适合 demo 和短视频

```
# Gemini 2.5 Pro · 原生视频输入
import google.generativeai as genai
from google.generativeai.types import File

genai.configure(api_key="...")
model = genai.GenerativeModel("gemini-2.5-pro")

# 1. 上传视频(支持 < 2 小时 / < 2GB)
video_file = genai.upload_file(path="meeting.mp4")
import time
while video_file.state.name == "PROCESSING":
    time.sleep(2)
    video_file = genai.get_file(video_file.name)

# 2. 直接问问题 · Gemini 内部抽帧 + ASR
response = model.generate_content([
    video_file,
    "这段会议讨论了几个主题?每个主题大概几分时讨论?\n"
    "请按时间戳格式输出:\n[00:00-05:30] 主题 A: ..."
])
print(response.text)

# 3. 跟 RAG 流水线整合
# · 拿到 LLM 输出的时间戳 + 主题
# · 转成 VideoSegment 入库
# · 检索时拿 segment_ts 跳转视频
```

老司机说

Gemini 2.5 视频优势

- 原生输入 · 无需 ffmpeg 拆
- 2M context · 跨小时长视频
- \$1.25/M tok · 1h 视频 ~\$1
- 内部自动抽帧 + ASR

何时用 Gemini 原生

- Demo / 快速 PoC
- 短视频 < 10 min
- 不需要 evidence 跳转

何时用三轨自建

- 长视频 + 需精确 evidence
- 中文 ASR 比 Gemini 强
- 自控数据隐私

2026 共识 · 混用

- Gemini · 快速 PoC
- 三轨自建 · 生产

CLIP-STYLE EMBEDDING

CLIP-style 跨模态 Embedding · 图 + 文同一空间

原理 · 训练时强制「匹配的图文 embedding 接近」 · 推理可以以图搜文 / 以文搜图

模型	类型	关键特性 2026	价格	适用
OpenAI CLIP	开源 (2021)	经典 · ViT-L/14 · 英文为主	\$0	入门 / 教学
SigLIP	开源 (Google 2023)	Sigmoid loss 替代 InfoNCE · 效果更好	\$0	英文 + 通用
Jina-CLIP v2	开源 + 商业	多语言 (89 语) · Late Interaction 支持	\$0.02/M tok	中文友好 ★
BGE-VL	开源 (BAAI)	中文场景 + 中英对齐	\$0	中文专 ★
Cohere Embed v4	商业 SaaS	图文统一 + Multilingual	\$0.13/M tok	商业 + 合规

```
from sentence_transformers import SentenceTransformer; from PIL import Image
model = SentenceTransformer("jinaai/jina-clip-v2")
text_emb = model.encode(["A photo of a cat in a garden"])
image_emb = model.encode([Image.open("cat.jpg")])
similarity = (text_emb @ image_emb.T) # 跨模态相似度
```

MULTIMODAL RAG ARCHITECTURE

Multimodal RAG 3 架构 · Hybrid 是 2026 生产推荐

不要纠结「Unified embedding」· 混用才是稳的

A · 全转文本

- PDF / 图 / 音视频 → markdown / 文字
- 一个 vector store 存全部
- 简单 · 跟 L01-L04 范式同
- 丢图细节 · 无 cross-modal 搜
- 场景 · 早期 + 文为主

B · 多 Modal Store ★

- 文 / 图 / 音独立 store
- Query 时各 store 并行检索
- 结果一起喂 multimodal LLM
- 灵活 + 模态间不污染
- 场景 · 2026 生产推荐 ★

C · Unified Embedding

- CLIP / SigLIP 统一空间
- 图 + 文同 store
- 跨模态搜 · 以图搜文
- cost 高 + 效果不如专 model
- 场景 · 电商 / 以图搜图

```
# 架构 B Hybrid · Type Router → 各 store 检索 → multimodal LLM 整合
type_ = await classify(query) # text / image / cross_modal
chunks = await {"text": text_rag, "image": image_rag, "cross_modal": cross_rag}[type_](query)
answer = await multimodal_llm.generate(query, chunks) # Claude / Gemini / GPT-5
```

COLPALI · 2024-2025

ColPali · 把整页 PDF 当图像检索 · 完全跳过解析

2024.7 论文 → 2025 工业落地 · 视觉文档检索的范式革新

原理 · PDF 不解析 · 整页截图当 image · VLM embedding 检索

传统 · PDF → Marker → markdown → chunk → text embedding → 检索。
ColPali · PDF 每页直接截图 → PaliGemma + Late-Interaction patch embedding → ColBERT-style MaxSim 检索。
效果 · 论文场景 ViDoRe benchmark · NDCG@5 大幅超过传统流水线 · 不必担心 IDP 解析错误 · 整页视觉一起检索。

版本	基础模型	关键能力	场景	2026 状态
ColPali	PaliGemma-3B	英文 PDF / 论文 · ViDoRe SOTA	英文学术	开源主流 ★
ColQwen2	Qwen2-VL-2B	中文 PDF · 2025 商业落地友好	中文 / 多语言 ★	中文首选
ColInternVL	InternVL-2	通用 · 学术活跃	研究	探索性

ANTI-PATTERNS

多模态处理 5 个反模式 · 踩过的坑

反模式 1(直接上 VLM)是 2024-2025 最高发 · 多花 10x 成本 · 效果还更差

反模式	具体表现	后果	正确做法
直接上 VLM 处理 PDF	所有 PDF 都截图给 Claude / GPT	cost 10-30x · 中文 OCR 更差	IDP 主干 + VLM 兜底
音频不 Diarization	多人会议混成一段文字	客户问追溯找不到说话人	pyannote 必上 / SaaS Speaker
视频不切 chunk	一长串 transcript 没法用	检索召回质量极差	按话题 / 场景切 chunk
图 embedding 用 text	图片描述塞进 text store	跨模态搜效果差	CLIP / SigLIP / Jina-CLIP
不存原文件	只存 markdown · 删原 PDF	客户问追溯回不去原文	原文件 + chunk 并存 versioning

踩坑提醒 1 那个真实账单 · 客户 10 万页 PDF · 直接 Claude Vision · 月账单 \$40K。
切到 Marker + Claude 兜底 5% 疑难 · 月账单降到 \$3.5K · 效果还好 · 这是 2026 多模态的 ROI 头号优化

MULTIMODAL FRONTIER 2025

2024-2025 多模态前沿 · 4 个值得关注的方向

共识 · 经典管道 + LLM 兜底 + 时间戳对齐 · 比花哨方案稳得多

Whisper-Large-v3-Turbo

2024.10

OpenAI 出 · 比 v3 快 8x · 精度只降 1%。
生产首选 · 中文 + 多语言 · 比 SaaS 便宜。

ColPali → ColQwen2

2024.7 → 2025

视觉文档检索 · 跳过 IDP · ViDoRe SOTA。
2025 ColQwen2 出来中文也能用 · 工业落地。

Gemini 2.5 原生视频

2024-2025

2M context · 原生视频输入 · 1h 视频 ~\$1。
2025 主流 VLM 全在跟进 · 快速 PoC 神器。

Claude Visual PDF

2025.3

Claude Sonnet 4.5 原生 PDF · layout-aware 内建。
IDP 边界在被 VLM 蚕食 · 兜底 5% 疑难场景。

COST MATH

多模态成本账 · 1 小时音频 / 1 小时视频真实账单

Demo 不算账 · 上线后账单一来吓死人 · 这张表帮你提前算清

任务	方案	成本	延迟	备注
1h 音频 ASR	Whisper-Turbo 自托管 (A10 GPU)	\$0.08	~5 min	\$0.4/GPU-hr
1h 音频 ASR	AssemblyAI / Deepgram SaaS	\$0.5-1.5	~3 min	开发省事
1h 音频 Diarization	pyannote 3.1 自托管	\$0.02	~3 min	同 GPU 跑
1h 视频 KeyFrame	ffmpeg + scene-detect	\$0.001	~1 min	CPU 即可
1h 视频 600 帧 Caption	Claude Sonnet 4.5	\$3-5	~5 min	\$0.005-0.008/img
1h 视频 Gemini 原生	Gemini 2.5 Pro	\$1	~2 min	\$1.25/M tok
100 页 PDF 整本	Marker (开源) + GPU	\$0.1	~3 min	A100 自托管
100 页 PDF Claude Vision	Claude Sonnet 4.5 全截图	\$15-30	~5 min	反模式 1 警示

已经造好了一座非结构化数据工厂

PDF / 网页 / 音频 / 视频 / 代码 · 全部以同一套资产化范式接入 · 下周 Week 8 RAG 服务化直接消费

L01 · Parse

IDP + 工具栈 2026
Marker / LlamaParse
Mistral OCR + 表格 3 策略

L02 · Chunk

Structure-aware + Late
Chunking
Contextual Retrieval
代码 AST + TreeSitter

L03 · Evidence

4 levels + EvidenceAnchor
Anthropic Citations API
L3 是 To B 及格线

L04 · Quality

4 维质检 + CI 门禁
Drift Detection 3 监控点
增量更新 chunk 级

L05 · Multimodal

三轨 + Vision LLM 横评
Whisper-Turbo + ColPali
Multimodal RAG Hybrid

本周交付物 · 全部 push 至 GitHub omnisupport-copilot 仓库

- pipelines/parse/marker_pipeline.py · Marker + Docling 主干
- pipelines/parse/table_extractor.py · 表格 3 策略
- pipelines/chunker/late_chunking.py · Jina v3 Late
- pipelines/chunker/code_ast.py · TreeSitter AST
- pipelines/quality/report.py · CI 门禁 quality report
- pipelines/incremental/update.py · content hash 增量
- pipelines/video/pipeline.py · 三轨 + segment_ts
- pipelines/parse/pdf_typer.py · PDF 类型识别
- pipelines/chunker/structure_aware.py · 结构感知切片
- pipelines/chunker/contextual.py · Anthropic Contextual
- services/rag_api/models.py · EvidenceAnchor schema
- pipelines/quality/drift_detector.py · 3 监控点
- pipelines/audio/process.py · Whisper + Diarization
- pipelines/multimodal/clip_embed.py · Jina-CLIP / BGE-VL